Aula 5

Modelagem de Processos e Negócios

Profª Adriana Bastos da Costa

Conversa Inicial

UML - *Unified Modelling Language*

- Nesta aula, vamos conversar sobre a importância de documentar em detalhes os requisitos de um *software*, principalmente a necessidade de descrever claramente as regras de negócios e as restrições existentes nessas regras para que o *software* funcione corretamente para atender aos usuários

- Tema 1 – Introdução e importância de restrições e padrões de negócio na engenharia de *software*
- Tema 2 – Identificação e documentação de restrições
- Tema 3 – Categorias de regras de negócio
- Tema 4 – Modelagem de restrições e tipos de padrões
- Tema 5 – Gabaritos de padrões de negócio

Introdução e importância de restrições e padrões de negócio na engenharia de *software*

Restrições e padrões de negócio

- A implementação de restrições e padrões de negócio é crucial para garantir que o sistema atenda aos objetivos empresariais, regulatórios e operacionais
- É preciso documentar os requisitos de um *software*, mas tão importante quanto entender os requisitos é identificar as restrições que afetam esses requisitos

Restrições de negócio

- São limitações ou condições específicas que devem ser consideradas e cumpridas
- Podem vir de diversas fontes, como requisitos de negócios, técnicos, regulamentações legais ou condições operacionais
- Geralmente não são negociáveis

Restrições de negócio

- Regulatórias: requisitos impostos por leis e regulamentos
- Financeiras: custo que não pode ser excedido
- Tecnológicas: tecnologias que devem ou não ser usadas
- Operacionais: tempo de resposta e disponibilidade do sistema, por exemplo

Padrões de negócio

- São diretrizes, regras e melhores práticas que orientam os desenvolvedores na criação do *software* de forma consistente, eficiente e de alta qualidade
- Os padrões cobrem diversas áreas do ciclo de vida do *software*, incluindo codificação, *design*, testes, documentação e manutenção

Padrões de negócio

- Padrões de qualidade: diretrizes que asseguram a qualidade do *software*, por exemplo, ISO/IEC 25010 para qualidade de produto de *software*, CMMI ou MPS.Br, como modelos de melhores práticas para construção de *software*

Padrões de negócio

- Padrões de desenvolvimento: uso de metodologias ágeis, padrões de codificação pré-definidos, linguagem a ser utilizada, padrões de arquitetura, entre outros
- Padrões de segurança: requisitos para garantir a segurança do *software* como, por exemplo, a ISO/IEC 27001, que é um padrão internacional para a gestão da segurança da informação

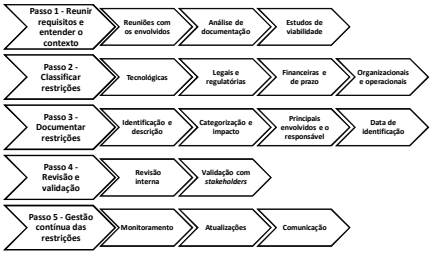
Premissas

- São suposições consideradas verdadeiras para fins de planejamento e execução do projeto
- Servem como base para tomar decisões e estabelecer expectativas
- São críticas para o planejamento de projetos, pois ajudam a definir o contexto no qual o *software* será desenvolvido e operado

Identificação e documentação de restrições

13/34

Processo de gestão das restrições



14/34

OCL - *Object Constraint Language*

- **Linguagem de especificação utilizada para definir restrições em modelos UML. Envolve a definição de regras e restrições que os objetos devem seguir**
- **Permite que os analistas especifiquem restrições textuais que podem ser verificadas automaticamente em modelos durante o processo de desenvolvimento de *software***

15/34

Exemplo 1:

"O saldo de uma conta bancária não pode ser negativo."

Contexto ContaBancaria
Inv SaldoNaoNegativo: self.saldo >= 0

Neste exemplo, "context ContaBancaria" define que a restrição se aplica à classe ContaBancaria e "inv SaldoNaoNegativo" é o nome da restrição. A expressão "self.saldo >= 0" especifica que o saldo da conta bancária (representado por "self.saldo") deve ser maior ou igual a zero. Essa restrição pode ser verificada automaticamente no modelo para garantir que o saldo nunca seja negativo.

16/34

Exemplo 2:

"Para um pedido ser considerado válido, a data de entrega deve ser posterior à data atual."

Contexto Pedido
inv DataEntregaPosteriorAtual: self.dataEntrega > OclAny.currentDate()

Nesse exemplo, a restrição "DataEntregaPosteriorAtual" é aplicada à classe Pedido. A expressão "self.dataEntrega > OclAny.currentDate()" garante que a data de entrega do pedido seja posterior à data atual, verificando automaticamente essa condição durante o desenvolvimento do software.

17/34

Categorias de regras de negócio

18/34

Regras de negócio

- São declarações específicas que definem ou restringem aspectos de uma empresa, influenciando comportamentos e decisões
- Elas orientam como o *software* deve operar para atender às necessidades, garantindo que suporte os processos de negócios de maneira eficaz e conforme as políticas da empresa

19/34

Principais características

Clareza	• Claramente definidas • Específicas para evitar ambiguidades
Independência	• Descrevem *o que* deve ser feito, não *como* • Independem da solução técnica
Mensuração e teste	• Devem poder ser verificadas e testadas
Derivadas das políticas e objetivos de negócio	• Podem ser derivadas das políticas, metas e regulamentos que a organização deve seguir

20/34

Principais categorias

Regras de definição	• "Um cliente ativo é definido como um cliente que realizou pelo menos uma compra nos últimos 12 meses."
Regras de restrição	• "Os pedidos que excedem R$ 10.000,00 devem ser aprovados por um gerente."
Regras de ação/ procedimento	• "Se um pagamento não for recebido dentro de 30 dias, enviar um aviso de pagamento atrasado ao cliente." "Enviar um e-mail de agradecimento automático para cada cliente que fizer uma compra superior a R$ 500,00."
Regras de inferência	• "Se um cliente faz mais de três reclamações em um mês, classifique-o como um cliente de alto risco."
Regras de cálculo	• "Calcular o desconto anual do cliente com base no total de compras realizadas no ano corrente."
Regras de validação	• "O campo 'data de nascimento' deve conter uma data válida e não pode estar no futuro."
Regras de autorização	• "Somente gerentes podem aprovar descontos superiores a 20%."

21/34

Regras de negócios

- As regras de negócios variam de empresa para empresa, pois elas refletem a forma como a empresa estrutura seus processos e suas políticas

22/34

Exemplo 1: sistema de gestão de vendas

- Regra de Restrição: "Um representante de vendas não pode conceder um desconto superior a 20% sem aprovação do gerente."
- Regra de Definição: "Um 'cliente VIP' é um cliente que gasta mais de R$ 50.000,00, anualmente."
- Regra de Ação: "Enviar um e-mail de agradecimento automático para cada cliente que fizer uma compra superior a R$ 500."

23/34

Exemplo 2: sistema bancário

- Regra de Restrição: "Transações superiores a R$10,000 devem ser reportadas ao departamento de *compliance*."
- Regra de Definição: "Uma 'conta inativa' é uma conta que não teve transações nos últimos 24 meses."
- Regra de Cálculo: "Calcular os juros mensais das contas de poupança com base na taxa de juros anual dividida por 12."

24/34

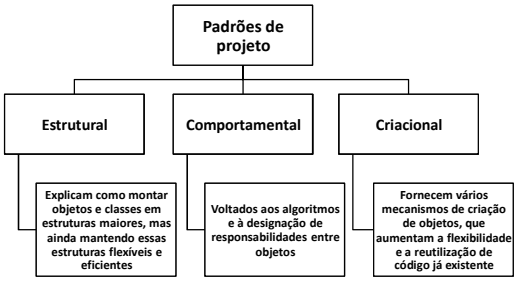
Modelagem de restrições e tipos de padrões

Modelagem de restrições e tipos de padrões

- Referem-se ao processo de definir e aplicar restrições e padrões de *design* para garantir a consistência, a qualidade e a eficiência do *software*
- Isso envolve identificar as restrições que o sistema deve seguir e os padrões de *design* que serão aplicados para resolver problemas comuns de forma eficaz

Padrões de projeto ou *design patterns*

- Em *software*, percebemos que determinados problemas são recorrentes, e se repetem mesmo em projetos diferentes
- É possível utilizar esse conhecimento para resolver o problema atual. É como se houvesse um padrão para solucionar certos tipos de problemas encontrados ao longo do desenvolvimento dos *softwares*



Padrões de arquitetura

- São peças fundamentais para concretizar toda a abstração e a escolha da estrutura da solução técnica adequada ao modelo de negócio
- É possível trabalhar com padrões de arquitetura diferentes no mesmo *software*, seja para atender determinados requisitos do negócio, facilitar a manutenção ou aproveitar vantagens de diferentes tecnologias

Padrões de arquitetura

- *Model-View-Controller* (MVC) – separa a aplicação em três componentes distintos: modelo, visão e controlador
- *Microservices* – arquitetura que organiza uma aplicação como um conjunto de serviços pequenos e independentes

Gabaritos de padrões de negócio

Gabaritos de padrão de negócio

- São modelos ou exemplos predefinidos que representam as melhores práticas e soluções para problemas comuns em determinado domínio de negócios. Eles são utilizados para orientar e agilizar o desenvolvimento de *software*, fornecendo diretrizes claras e estruturadas para a implementação de funcionalidades específicas

Gabaritos de padrão de negócio

- Podem incluir modelos de processos, regras de negócio, fluxos de trabalho, entre outros elementos que ajudam a padronizar e otimizar a maneira como as atividades de negócio são realizadas

Gabaritos de padrão de negócio

- São soluções generalistas para problemas recorrentes durante o desenvolvimento de um *software*
- Não se trata de um *framework* ou um código pronto, mas de uma definição de alto nível de como um problema comum pode ser tecnicamente solucionado